

ETX 方案 v2：在 MoE 解码上超过 vLLM

作者：Kailun Chen · 状态：v2 计划，取代 PLAN-fusion-win.md · 日期：2026-09-25 · 依据：设计文档 15.5–15.7 节，examples/llm/results/

0. 结论先行

- **换目标：** 不再在 batch=1 的 dense 解码上证明融合。实测证明那是一条没有可重叠空间的屏障链，megakernel 最多和 HIP Graph 持平。
- **新目标：** Qwen3-30B-A3B (128 专家选 8) 单卡 MI300X 解码，batch=1 时每 token 不超过 **3.5 ms**，vLLM 最强配置是 4.84 ms；同时在 batch 4–8 上保持领先。全程逐 token 与 HF 一致。
- **为什么是 MoE：** vLLM 在这个模型上只跑到带宽的 26%（每 token 读 6.1 GB 用了 4.84 ms，带宽下限约 1.5 ms），在 dense 8B 上却有 60%。差距来自路由之后才能选专家、专家矩阵小而碎、依赖路由结果的 kernel 固化不进 Graph。这三条正是 megakernel 的长项。
- **两个独立问题分开解：** 融合的收益只在 MoE 结构上找；tile 的速度单独做，目标是 GEMV 3 TB/s、attention 每层 5 μ s 以内。
- **预算：** 4 个阶段，约 6 次 GPU 会话，1x MI300X 每小时 \$2.99，合计约 \$25–35。余额 \$22.37。

1. 已经确认的事实

事实	依据
同一套 tile，融合版和 HIP Graph 不融合版在 dense 上持平（差 2–5%）	分片图实测，三个模型
与 vLLM 的 2 倍差距全在 tile：GEMV 1.5–2.5 TB/s，attention 加合并每层 50 μ s	Qwen3-8B 逐阶段 trace
用好 tile 时编译出的 kernel 能超过 vLLM 8%、接近手写版 6%	DeepSeek, fleet 的 tile
vLLM 在 MoE 上只用到 26% 带宽	4.84 ms 对 6.1 GB
每 CU 一个 workgroup 对持久化 kernel 更好；L2 预热式预取是负收益；relay 的分层 acquire 不安全	2026-09-24/25 实测

2. MoE 解码的一层长什么样

现在分片图下 Qwen3-30B-A3B 每层 219 μ s，各阶段（每 CU 一个 workgroup）：

阶段	现在	目标	手段
qkv	22 μ s	10	GEMV tile
post (q/k norm、RoPE、写 KV)	15	0	并进 qkv 收尾
attention	45	10	flash-decode 写法
merge	22	0	由每头最后完成的 attention 任务顺手做
o_proj	21	5	小 K 的 GEMV tile
fold + router	15	8	已是 XCD 本地
专家 gate_up	51	18	专家 GEMV tile
专家 down	34	12	专家 GEMV tile

阶段	现在	目标	手段
2 次整卡交接	6	6	—
合计	219	约 70	48 层约 3.4 ms, 加 lm_head 0.2 ms

MoE 结构上的融合收益已经在分片图里：router 每个 XCD 各算一份，router 到专家没有整卡屏障，XCD k 一算出自己的专家就开始读它的权重。vLLM 这里是 router、top-k、fused_moe 三个 kernel 加边界，而且 fused_moe 的分组 GEMM 是为大 batch 设计的，单 token 效率低。

3. 阶段

阶段 1：tile 提速（2–3 次会话）

- **GEMV**：每 CU 一个 workgroup 时寄存器预算 512，在途读请求提到 32–48 个；每个任务的权重块提到 128 KB 以上，小矩阵（o_proj、router）改为多任务合并；输入向量在 LDS 用 bf16 存，HF 的线性层输入本来就是 bf16，数值不变。目标 3 TB/s。
- **attention**：4 个 wave 各管一段位置，128 维向量化点积，输出部分和；每头最后完成的分块直接合并，删掉 merge 阶段。
- **post 并进 qkv 收尾**：qkv 任务按整数个头切行，收尾直接做 norm、RoPE 和写 KV。
- **验收**：Qwen3-30B-A3B 融合版不超过 5 ms（追平 vLLM），Qwen3-8B 不超过 6 ms；逐 token 与 HF 一致。

阶段 2：MoE 结构收益（1–2 次会话）

- **专家权重预取进 LDS**：XCD 的 fold 任务算出路由后，专家 gate_up 任务在等本地事件时把自己那段权重读进 LDS 暂存区（每 CU 剩余约 30 KB），不再用 L2 预热。
- **top-k 与 XCD 数不等时的分配**：专家按负载分给 XCD，不再一个槽位一个 XCD。
- **验收**：Qwen3-30B-A3B 不超过 3.5 ms，超过 vLLM 25% 以上。

阶段 3：小 batch（1–2 次会话）

- batch 4–8：同一步里多个 token 的专家按专家分组，用 ETX 的数据相关边（"b->topk[b,:]"、按 indptr 的 range 映射）表达，同一专家的权重只读一次。
- 阶段交接的等待由别的 token 的任务填上，这是 Graph 做不到的。
- **验收**：batch 4 和 8 上每 token 时间不超过 vLLM 同 batch 的 80%。

阶段 4：对比与交付（1 次会话）

- Qwen3-30B-A3B × {vLLM 最强、ETX 融合、ETX 不融合} × batch {1, 4, 8}，1024 上下文，各 3 轮，逐 token 对 HF。
- 更新设计文档、精简版 PDF、README。

4. 风险

- **专家 GEMV 到 3 TB/s 是估计**。单个专家只有 3 MB，分到 38 个 worker 每任务 80 KB，延迟隐藏更难；达不到就退到 2 TB/s，对应约 4.3 ms，仍比 vLLM 快 11%。
- **LDS 预取占空间**。attention 和 GEMV 的输入向量已占用大半 LDS，预取深度要用 A/B 决定。

- 小 batch 的路由分组要改编译器的实例化路径。这部分代码有过 MoE 例子验证，但没在真实模型上跑过。
- GPU 可用性。近几天每次要等几小时才有空位。

5. 验证方法（2026-09-25 起）

- **CPU 参考实现** `examples/llm/ref_numpy.py`：用 numpy 按 tile 的运算顺序和 bf16 舍入位置重写整条流水线，本地对 Qwen2.5-0.5B 与 HF 逐 token、逐层比对（8/8 token 一致，每层余弦大于 0.9999），一次 3.4 秒。tile 的算法改动先在这里验证，GPU 只量速度。
- **逐层导出**：GPU 端 `run --dump` 把每层输入的残差写出来，`compare_dump.py` 和参考实现逐层比，一次运行就能定位到偏差出现在哪一层。
- **GEMV 微基准** `bench/gemv_bench.hip`：和 tile 共用 `examples/llm/gemv.h`，几秒扫一遍任务大小和 K，tile 调优不再跑整模型。

6. 顺序

先做阶段 1。它同时回答两件事：tile 能不能到 3 TB/s，以及到了以后 MoE 融合版是否已经追平 vLLM。阶段 1 过线，阶段 2 的预取和分配决定能超多少；阶段 1 不过线，问题就明确在 tile 工程上，可以考虑接入外部 tile。